



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Bernardette Sibila Aragón Leóm Carné: 1265425

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Una API REST (interfaz de programación de aplicaciones de transferencia de estado representacional) es un estilo de arquitectura que se considera el estándar para diseñar y crear las aplicaciones en red que impulsan la web. Proporciona un conjunto de reglas y restricciones que, si se siguen, dan como resultado servicios web sencillos, escalables y fáciles de integrar.

2. ¿Qué es un endpoint?

Es una dirección digital (URL) en programación o un dispositivo físico en redes que permite la comunicación entre sistemas.

3. ¿Qué diferencia existe entre una URL y un endpoint?

Una URL es la dirección general de un recurso en internet, mientras que un endpoint es una URL específica combinada con un método HTTP que le indica a una API qué acción realizar sobre un recurso.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

- GET: Obtiene información o datos de un servidor sin alterar su estado. Es un método de solo lectura.
- POST: Envía datos al servidor para crear un nuevo recurso. Cada vez que se ejecuta, puede generar una nueva entrada.
- PUT: Reemplaza por completo un recurso existente con la nueva información enviada. Si el recurso no existe, puede llegar a crearlo.
- PATCH: Realiza actualizaciones parciales en un recurso. Modifica únicamente los campos específicos que se envían en la petición.
- DELETE: Solicita la eliminación de un recurso del servidor

5. ¿Qué es JSON y por qué es común en las APIs?

JSON (JavaScript Object Notation) es un formato utilizado para representar y transportar información de manera estructurada. Es común en las APIs porque es sencillo de leer y escribir, tanto para las personas como para los programas, y permite intercambiar datos fácilmente entre diferentes sistemas.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Un código de estado HTTP es un número que indica el resultado de una solicitud realizada a un servidor mediante HTTP. Algunos ejemplos son:

- **200:** La solicitud se realizó correctamente.
- **404:** El recurso solicitado no fue encontrado.
- **500:** Ocurrió un error interno en el servidor.

7. ¿Qué información se puede encontrar en la respuesta de una API?

En la respuesta de una API se puede encontrar información como el código de estado HTTP, los datos solicitados, mensajes de respuesta, errores y diferentes encabezados HTTP. Los datos normalmente se encuentran en formatos como JSON.

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

Postman permite probar las solicitudes y respuestas de una API sin necesidad de desarrollar primero toda la aplicación. Esto facilita comprobar si los endpoints funcionan correctamente, revisar los datos que devuelve el servidor, detectar errores y verificar los diferentes métodos HTTP, como GET, POST, PUT y DELETE.

9. ¿Qué es y para qué sirve Swagger UI?

Swagger UI es una herramienta que permite visualizar y probar la documentación de una API desde una interfaz



web. Muestra información sobre los endpoints disponibles, los parámetros que reciben, las respuestas que pueden generar y permite realizar solicitudes directamente desde la misma interfaz.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Postman sirve para enviar solicitudes a una API y analizar las respuestas que devuelve el servidor. Se utiliza para probar diferentes endpoints, métodos HTTP, parámetros y datos enviados, ayudando a comprobar el funcionamiento de la API y a encontrar posibles errores durante su desarrollo.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.

```
https://petstore3.swagger.io/api/v3/pet/1
```

- Indique el método HTTP utilizado.
GET
- Indique qué parámetro necesita la solicitud.
PET ID
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
Código recibido: 200
- Explique con sus palabras qué información contiene la respuesta.
La respuesta muestra la información de la mascota que se consultó mediante su ID. En este caso, se consultó la mascota con ID 1 y la API devolvió sus datos en formato JSON. Se puede observar que su nombre es "Pet1", tiene dos fotografías, no tiene etiquetas y su estado actual es "available". Además, se recibió el código 200, lo que indica que la consulta se realizó correctamente.

Evidencia: captura de Swagger mostrando la operación y su respuesta.



3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.



Creación de mascota

MIA_Laboratorio_No.1_Semana... Swagger UI

petstore3.swagger.io/#/pet/addPet

Parameters Cancel Reset

No parameters

Request body required application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 12654,
  "name": "Michi",
  "category": {
    "id": 2,
    "name": "Cats"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "domestico"
    }
  ],
  "status": "available"
}
```

Execute Clear

Responses

23°C Mayorm: nublado

Buscar

10:16 p. m. 02/10/2026

Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 12654,
    "name": "Michi",
    "category": {
      "id": 2,
      "name": "Cats"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 1,
        "name": "domestico"
      }
    ],
    "status": "available"
  }'
```

Request URL

<https://petstore3.swagger.io/api/v3/pet>

Server response



The screenshot shows the Swagger UI interface in a web browser. The URL bar displays `petstore3.swagger.io/#/pet/addPet`. The 'Request URL' field contains `https://petstore3.swagger.io/api/v3/pet`. The 'Server response' section shows a 200 status code. The 'Response body' is a JSON object:

```
{  "id": 12654,  "category": {    "id": 2,    "name": "Cats"  },  "name": "Michi",  "photoUrls": [    "string"  ],  "tags": [    {      "id": 1,      "name": "domestico"    }  ],  "status": "available"}
```

. The 'Response headers' section lists various headers including `access-control-allow-headers`, `access-control-allow-methods`, `access-control-allow-origin`, `access-control-expose-headers`, `content-length`, `content-type`, `date`, and `server`. The 'Responses' table at the bottom shows a 200 status code with the description 'Successful operation'.

Consulta de mascota por el método GET

The screenshot shows the Swagger UI interface in a web browser. The URL bar displays `petstore3.swagger.io/#/pet/getPetById`. The 'petId' field is marked as required and has the value `12654` entered. The 'Execute' button is highlighted. The 'Responses' section shows a 200 status code. The 'Request URL' field contains `https://petstore3.swagger.io/api/v3/pet/12654`. The 'Server response' section shows a 200 status code. The 'Response body' is a JSON object:

```
{  "id": 12654,  "category": {    "id": 2,    "name": "Cats"  },  "name": "Michi",  "photoUrls": [    "string"  ],  "tags": [    {      "id": 1,      "name": "domestico"    }  ],  "status": "available"}
```

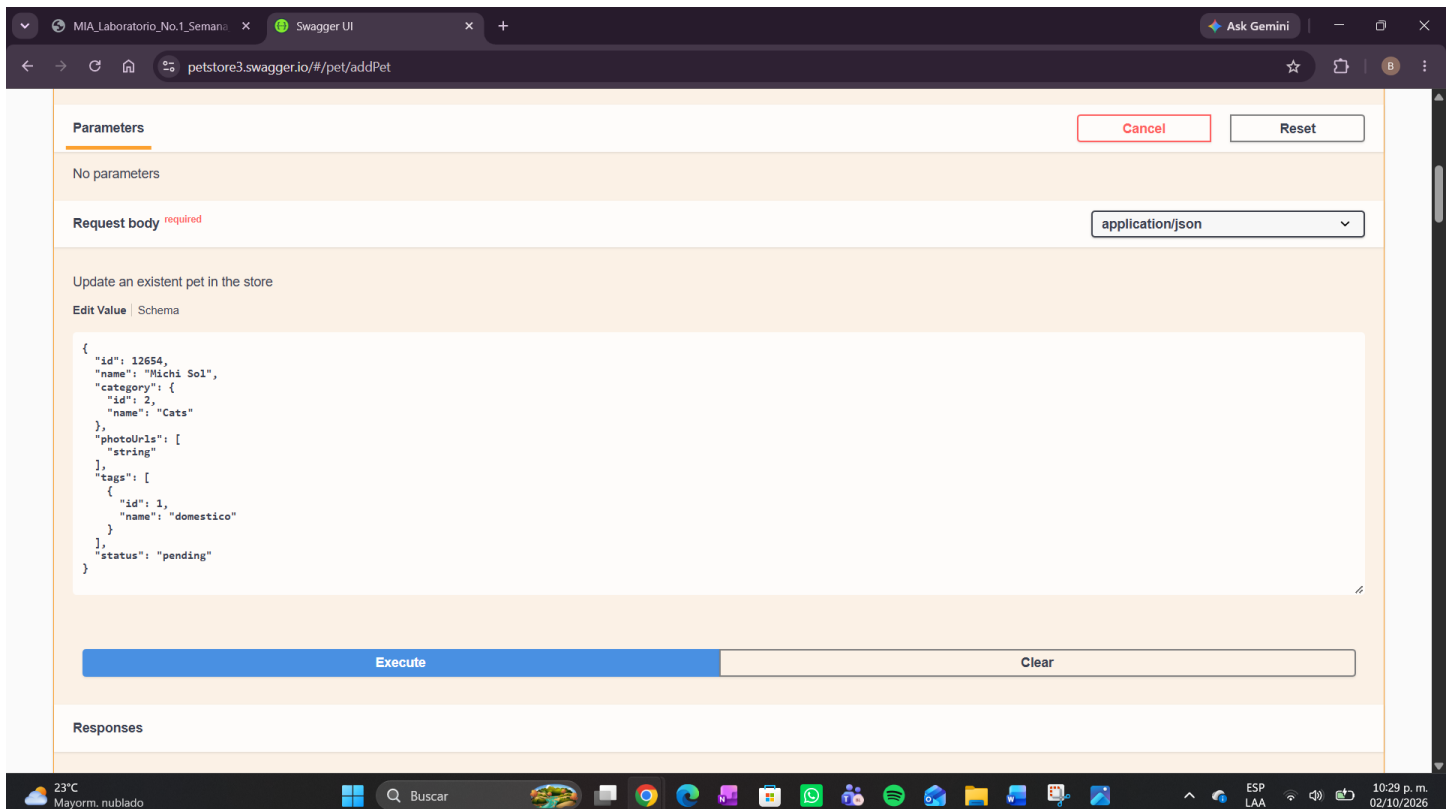
. The 'Curl' section shows the command:

```
curl -X 'GET' \  'https://petstore3.swagger.io/api/v3/pet/12654' \  -H 'accept: application/json'
```



4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
Se utilizó el método HTTP PUT porque permite actualizar un recurso que ya existe. En este caso, se utilizó para modificar la información de la mascota con ID 12654.
- ¿Qué datos fueron enviados en el body?
En el body se enviaron los datos de la mascota en formato JSON. Se mantuvo el ID 12654 y se modificaron el nombre a "Michi Sol" y el estado a "pending". También se enviaron los datos de su categoría, fotografías y etiquetas.
- ¿Qué código HTTP devolvió la API?
200
- ¿Cómo comprobó que el cambio realmente ocurrió?
Comprobé que el cambio realmente ocurrió realizando una nueva solicitud GET utilizando el ID 12654. La respuesta mostró el nombre "Michi Sol" y el estado "pending", que son los valores que había modificado mediante PUT. Por lo tanto, se pudo verificar que los cambios fueron guardados correctamente.





Responses

Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 12654,
    "name": "Michi Sol",
    "category": {
      "id": 2,
      "name": "Cats"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 1,
        "name": "domestico"
      }
    ],
    "status": "pending"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code	Details
200	Response body <pre>{ "id": 12654, "category": { "id": 2, "name": "Cats" }, "name": "Michi Sol", "photoUrls": ["string"], "tags": [{ "id": 1, "name": "domestico" }], "status": "pending" }</pre>

23°C Mayorm, nublado

10:29 p. m. 02/10/2026

petid * required ID of pet to return
integer(int64) (path) 12654

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/12654' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/12654

Server response

Code	Details
200	Response body <pre>{ "id": 12654, "category": { "id": 2, "name": "Cats" }, "name": "Michi Sol", "photoUrls": ["string"], "tags": [{ "id": 1, "name": "domestico" }], "status": "pending" }</pre> Download

Response headers

23°C Mayorm, nublado

10:33 p. m. 02/10/2026



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
El parámetro utilizado es status, que permite especificar el estado de las mascotas que se desean consultar.
- Explique dónde se envía ese parámetro.
El parámetro status se envía en la URL como un parámetro de consulta (query parameter). En esta solicitud se puede observar como ?status=available, indicando que se quieren consultar las mascotas cuyo estado es available.
- Observe la estructura de la respuesta.
La respuesta tiene una estructura de arreglo JSON, ya que contiene varias mascotas que cumplen con el filtro solicitado. Cada elemento del arreglo representa una mascota y contiene datos como su ID, categoría, nombre, fotografías, etiquetas y estado. En este caso, las mascotas mostradas tienen el estado available.
- Indique qué sucede si cambia el valor del parámetro por pending.
Al cambiar el valor del parámetro status de available a pending, la API filtra y devuelve únicamente las mascotas que tienen ese estado. En este caso, la respuesta fue exitosa con código HTTP 200 y se obtuvo un arreglo JSON con la información de las mascotas encontradas. Se puede observar que los elementos de la respuesta tienen el campo "status": "pending", lo que demuestra que el parámetro fue aplicado de forma correcta.



Available

The screenshot shows the Swagger UI interface for the 'petstore3.swagger.io' API. The endpoint selected is 'pet/findPetsByStatus'. The 'Parameters' section shows a required query parameter 'status' with a dropdown menu set to 'available'. Below the parameters, there are 'Execute' and 'Clear' buttons. The 'Responses' section shows a 200 status code with a JSON response body. The response body contains a single pet object with the following details: id: 4, category: Dogs, name: Dog 1, and two photo URLs.

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \
  -H 'accept: application/json'
```

```
https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available
```

```
{
  "id": 4,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Dog 1",
  "photoUrls": [
    "url1",
    "url2"
  ],
  "tags": [
  ]
}
```

Pending

The screenshot shows the Swagger UI interface for the 'petstore3.swagger.io' API. The endpoint selected is 'pet/updatePet'. The 'Parameters' section shows a required query parameter 'status' with a dropdown menu set to 'pending'. Below the parameters, there are 'Execute' and 'Clear' buttons. The 'Responses' section shows a 200 status code with a JSON response body. The response body contains a single pet object with the following details: id: 12654, category: Cats, name: Michi Sol, and one photo URL.

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending' \
  -H 'accept: application/json'
```

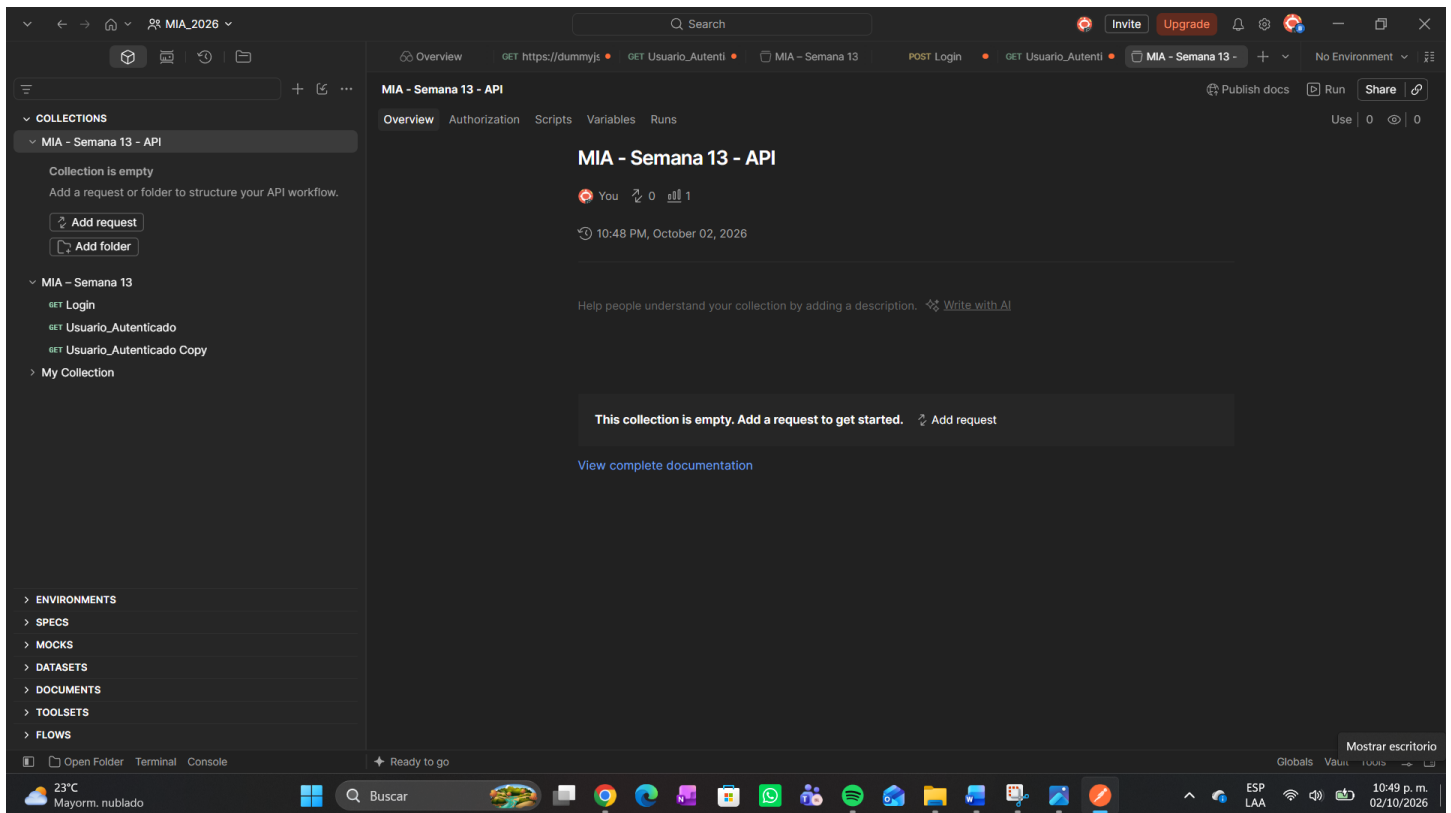
```
https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending
```

```
{
  "id": 12654,
  "category": {
    "id": 2,
    "name": "Cats"
  },
  "name": "Michi Sol",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Domestic"
    }
  ]
}
```



6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.





Screenshot of the Swagger UI interface showing a GET request to `https://petstore3.swagger.io/api/v3/pet/12654`. The response is a JSON object:

```
{  "id": 12654,  "category": {    "id": 2,    "name": "Cats"  },  "name": "Michi Sol",  "photoUrls": [    "string"  ],  "tags": [    {      "id": 1,      "name": "domestico"    }  ],  "status": "pending"}
```

The status is 200 OK. The interface also shows a sidebar with collections and environments.

Screenshot of the Swagger UI interface showing a POST request to `https://petstore3.swagger.io/api/v3/pet`. The response is a JSON object:

```
{  "id": 1001,  "name": "Michi Postman",  "category": {    "id": 3,    "name": "Cats"  },  "photoUrls": [    "foto1"  ],  "tags": [    {      "id": 1,      "name": "domestico"    }  ]}
```

The status is 200 OK. The interface also shows a sidebar with collections and environments.



Postman interface showing a PUT request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object:

```
1 {
2   "id": 1001,
3   "name": "Michi Postman Modificado",
4   "category": {
5     "id": 3,
6     "name": "Cats"
7   },
8   "photoUrls": [
9     "foto1"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "domestico"
15    }
16  ]
17 }
```

The response is a 200 OK status with a 447 ms response time and 519 B body size. The response body is a JSON object:

```
1 {
2   "id": 1001,
3   "category": {
4     "id": 3,
5     "name": "Cats"
6   },
7   "name": "Michi Postman Modificado",
8   "photoUrls": [
9     "foto1"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "domestico"
15    }
16  ]
17 }
```

Postman interface showing a GET request to `https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending`. The request parameters are:

Key	Value	Description	Bulk Edit
status	pending		

The response is a 200 OK status with a 364 ms response time and 1.39 KB body size. The response body is a JSON array:

```
1 [
2   {
3     "id": 3,
4     "category": {
5       "id": 2,
6       "name": "Cats"
7     },
8     "name": "Cat 3",
9     "photoUrls": [
10      "url1",
11      "url2"
12    ],
13     "tags": [
14       {
15         "id": 1,
16         "name": "domestico"
17       }
18     ]
19   }
20 ]
```



The screenshot shows the Postman interface with a DELETE request to `https://petstore3.swagger.io/api/v3/pet/1001`. The response is a 200 OK status with a body of `Pet deleted`. The left sidebar shows a collection named 'MIA - Semana 13 - API' with several endpoints. The bottom status bar shows the system time as 11:06 p.m. on 02/10/2026.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa



Consultar un ID inexistente

The screenshot shows the Postman interface with a GET request to `https://petstore3.swagger.io/api/v3/pet/9999`. The response is a `404 Not Found` status with a response time of 310 ms and a body of `Pet not found`. The left sidebar shows a collection named 'MIA - Semana 13 - API' with various endpoints. The bottom status bar shows the system clock as 11:07 p.m. on 02/10/2026.

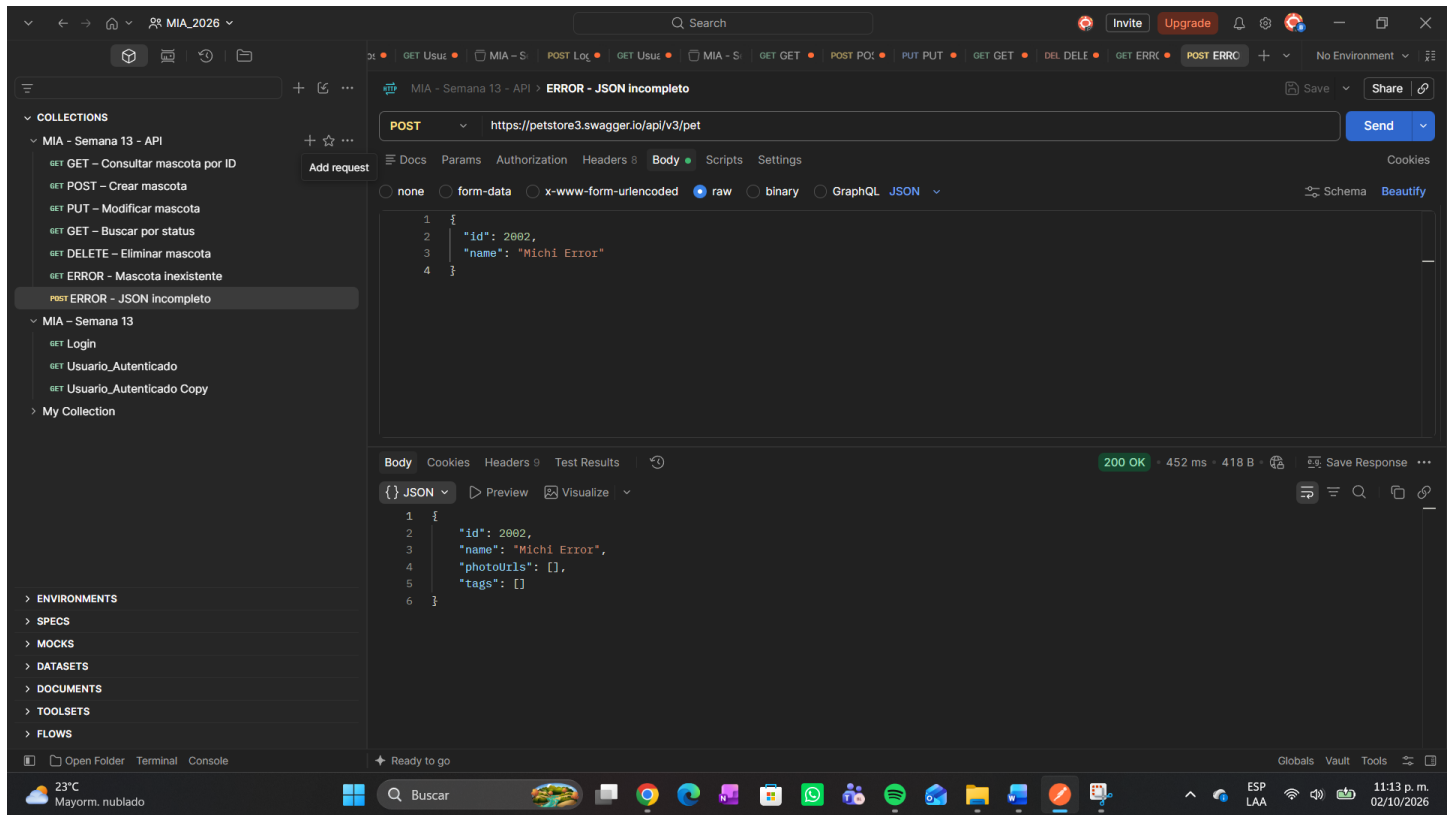
Key	Value	Description	Bulk Edit
Key	Value	Description	

Body	Cookies	Headers	Test Results
<pre>1 Pet not found</pre>			

- a) Solicitud realizada: GET /pet/9999
- b) Código HTTP recibido: 404 Not Found
- c) Respuesta de la API: Pet not found
- d) Qué esperaba que ocurriera: Esperaba que la API indicara que la mascota solicitada no existe
- e) Qué ocurrió realmente: La API respondió con el código 404 y el mensaje Pet not found
- f) Explicación de la causa: El ID 9999 no corresponde a una mascota existente en la API, por lo que el recurso no pudo ser encontrado



Enviar un JSON incorrecto o incompleto



- Solicitud realizada: POST /pet con un JSON incompleto
- Código HTTP recibido: 200 OK
- Respuesta de la API: La API aceptó el registro y devolvió los campos enviados junto con photoUrls y tags como arreglos vacíos
- Qué esperaba que ocurriera: esperaba que la API rechazara la solicitud debido a que el JSON no incluía todos los atributos utilizados normalmente para registrar una mascota
- Qué ocurrió realmente: La API aceptó la solicitud y creó el recurso, devolviendo el código 200
- Explicación de la causa: La API permite registrar una mascota utilizando únicamente algunos de los atributos del objeto, por lo que la ausencia de otros campos no fue considerada un error por el servidor



Utilizar un parámetro con un valor no esperado

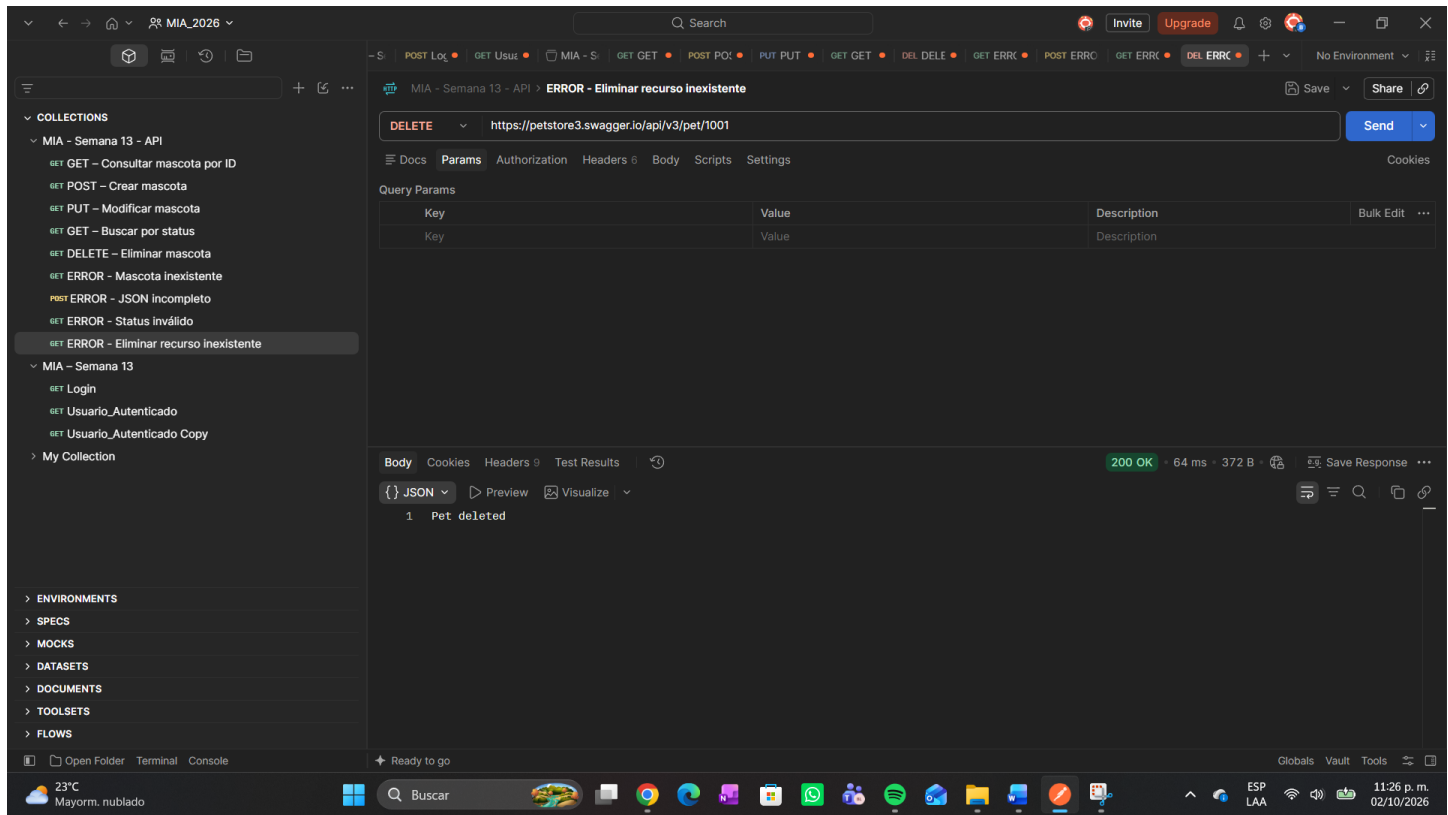
The screenshot shows the Postman interface with a collection named 'MIA - Semana 13 - API'. The selected request is 'GET - Status inválido' with the URL 'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=cualquiercosa'. The response is a 400 Bad Request with a status of 1.89 s and 505 B. The response body is JSON:

```
{  "code": 400,  "message": "Input error: query parameter 'status' value 'cualquiercosa' is not in the allowable values '[available, pending, sold]'"}
```

- Solicitud realizada: GET /pet/findByStatus?status=cualquiercosa
- Código HTTP recibido: 400 Bad Request
- Respuesta de la API: La API indicó que el valor cualquiercosa no se encuentra entre los valores permitidos: available, pending y sold
- Qué esperaba que ocurriera: Esperaba que la API rechazara la solicitud debido a que se utilizó un valor no permitido para el parámetro status
- Qué ocurrió realmente: La API respondió con el código 400 y mostró un mensaje indicando que el valor proporcionado no es válido
- Explicación de la causa: El parámetro status solamente acepta los valores available, pending o sold, por lo que cualquiercosa genera un error de solicitud



Intentar eliminar un recurso que ya no existe.



- g) Solicitud realizada: DELETE /pet/1001
- h) Código HTTP recibido: 200 OK
- i) Respuesta de la API: Pet deleted
- j) Qué esperaba que ocurriera: Esperaba que la API indicara que la mascota ya no existía o que devolviera un código de error
- k) Qué ocurrió realmente: La API respondió con el código 200 y el mensaje Pet deleted, aunque el recurso con ID 1001 ya había sido eliminado anteriormente
- l) Explicación de la causa: La API permite realizar nuevamente la operación de eliminación sobre el mismo ID sin devolver un error que indique que el recurso ya no existe

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.



Operación	Endpoint	Método HTTP	Respuesta que debe analizar el programa
Registrar mascota	/pet	POST	Código HTTP y datos de la mascota registrada
Consultar información	/pet/3003	GET	Código HTTP y datos de la mascota
Actualizar estado	/pet	PUT	Código HTTP y estado actualizado
Eliminar mascota	/pet/3003	DELETE	Código HTTP y mensaje de confirmación
Verificar que no existe	/pet/3003	GET	Código HTTP y mensaje de recurso no encontrado

- Ejecute el flujo completo en Postman.
Capture evidencia de cada paso.

Registro de mascota

The screenshot shows the Postman interface with a collection named 'MIA - Semana 13 - API'. The selected test is 'DETECTIVE - Registrar mascota'. The request is a POST to 'https://petstore3.swagger.io/api/v3/pet' with a raw JSON body. The response is a 200 OK status with a JSON body containing pet details.

```
POST https://petstore3.swagger.io/api/v3/pet

{
  "id": 3003,
  "name": "Michi Detective",
  "category": {
    "id": 2,
    "name": "Cats"
  },
  "photoUrls": [
    "foto1"
  ],
  "tags": [
    {
      "id": 1,
      "name": "doméstico"
    }
  ]
}
```

Body: JSON Preview Visualize

```
1 {
2   "id": 3003,
3   "category": {
4     "id": 2,
5     "name": "Cats"
6   },
7   "name": "Michi Detective",
8   "photoUrls": [
9     "foto1"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "doméstico"
15    }
16  ]
17 }
```

200 OK · 156 ms · 512 B · Save Response

Consulta de mascota



The screenshot shows the Postman interface with a GET request to `https://petstore3.swagger.io/api/v3/pet/3003`. The response is a 200 OK status with a JSON body containing pet information.

```
{  "id": 3003,  "category": {    "id": 2,    "name": "Cats"  },  "name": "Michi Detective",  "photoUrls": [    "foto1"  ],  "tags": [    {      "id": 1,      "name": "domestico"    }  ]}
```

Actualizar estado

The screenshot shows the Postman interface with a PUT request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object with pet details and a status of "pending". The response is a 200 OK status.

```
{  "name": "Cats",  "photoUrls": [    "foto1"  ],  "tags": [    {      "id": 1,      "name": "domestico"    }  ],  "status": "pending"}
```



Eliminar mascota

The screenshot shows a REST client interface with a sidebar on the left containing a list of collections. The main panel displays a DELETE request to the URL `https://petstore3.swagger.io/api/v3/pet/3003`. The response is a 200 OK status with a body of `{ "pet": "deleted" }`.

DELETE `https://petstore3.swagger.io/api/v3/pet/3003` **Send**

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 9 Test Results **200 OK** 157 ms 372 B Save Response

JSON Preview Visualize

```
1 Pet deleted
```

Open Folder Terminal Console + Ready to go

23°C Mayomr, nublado

11:37 p.m. 02/10/2026



Verificar inexistencia

The screenshot shows the Postman interface with a GET request to `https://petstore3.swagger.io/api/v3/pet/3003`. The response is a `404 Not Found` status with a body of `Pet not found`. The left sidebar shows a collection named "MIA - Semana 13 - API" with various endpoints. The bottom status bar shows the system clock as 11:38 p.m. on 02/10/2026.

Key	Value	Description	Bulk Edit
Key	Value	Description	

Key	Value	Description
Key	Value	Description

Body: 404 Not Found, 362 ms, 373 B

Continue in Agent Mode

Diagnose Decline

XML Preview Debug with AI

1 Pet not found

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.